

SUBASTA SEGURA: RETIRO, CEI Y GAS MEDIDO

Completar una subasta que custodia dinero ajeno, ver cómo un contrato hostil congela la versión ingenua, desplegarla en Sepolia y medir cuánto cuesta defenderse.

MODALIDAD	DURACIÓN	REQUISITO	META
En parejas en la sala	75 min en clase + informe en casa	Repositorio de la S5 + billetera del curso	23 pruebas en verde + subasta en Sepolia

CÓMO LEER ESTA GUÍA

Está escrita para que puedan hacer el laboratorio **sin ayuda**, incluso si se perdieron la clase. Cada paso dice qué escribir, qué deberían ver y cómo saber que funcionó. Si algo no sale como aquí se describe, vayan primero a la sección **13 · Errores frecuentes**. Las cifras de gas de esta guía no son estimaciones: se midieron con `npx hardhat test --gas-stats` sobre este mismo repositorio (Hardhat 3.16.0, Solidity 0.8.28, optimizador con 200 ejecuciones). La red local es determinista: ustedes deben obtener exactamente las mismas.

01 Qué se construye y qué se demuestra

[LEER PRIMERO](#)

Objetivo de la sesión (plan del curso): manejar estructuras compuestas y aplicar patrones seguros de manejo de fondos. **Resultado de aprendizaje:** RA3 (programar contratos aplicando patrones de diseño y estándares) y, de forma complementaria, RA5 (base de la seguridad que se profundiza en la Sesión 9).

El laboratorio tiene cinco partes, cada una atada a láminas concretas de la clase:

PARTE	QUÉ HACEN	LÁMINAS
1 · Completar	Seis TODO en <code>Subasta.sol</code> hasta que las 23 pruebas pasen.	A.1–A.4, A.12–A.16
2 · El ataque	Ver cómo <code>PostorHostil</code> congela la subasta push y no afecta la pull. Romper su contrato a propósito.	A.13, A.14
3 · Desplegar	Llevar la subasta a Sepolia desde Remix y pujar desde dos cuentas.	B.6
4 · Medir	Llenar la tabla comparativa de gas push frente a pull con dos escenarios.	A.17–A.20
5 · Cerrar	Finalizar la subasta en Sepolia y retirar. Escribir el informe de decisiones de diseño.	B.8

LA LECCIÓN EN UNA FRASE

Un contrato que envía dinero depende de que el otro quiera recibirlo. La versión «amable» de la subasta — devolver el dinero al instante— puede ser congelada por un contrato que **no roba nada**: solo se niega a recibir. La versión segura cuesta más gas. Ustedes van a demostrar las dos cosas con pruebas y con cifras propias.

02 Qué se entrega

EVIDENCIA DE LA SESIÓN

La evidencia que pide el plan para la Sesión 7 es **contrato desplegado + informe de decisiones de diseño**. En concreto, un solo PDF por pareja (máximo cuatro páginas con anexos) con:

1. Captura de la terminal con `23 passing` para `test/s07/Subasta.test.js` con su versión de `Subasta.sol`.
2. La **tabla de gas** de la sección 9, llena con sus cifras, y las respuestas a sus tres preguntas.
3. Dirección de su `Subasta` en Sepolia y los hashes de: dos pujas, la finalización y un retiro.
4. El **informe de decisiones de diseño** (sección 11), con al menos cinco decisiones.
5. Las seis respuestas de reflexión (sección 12).

Sin la evidencia en cadena (dirección y hashes), el laboratorio no se califica: es la política del curso.

03 Preparación desde cero

10 MINUTOS SI TODO ESTÁ

El laboratorio usa el repositorio único del curso, `laboratorios-evm`, que se instaló en la Sesión 5. Si ya lo tienen funcionando, comprueben las versiones y salten al paso 3.4.

3.1 • Comprobar Node.js y Git

Abran una terminal (en Windows: tecla Windows, escriban `PowerShell`, Enter) y escriban:

```
node --version
git --version
```

Salida esperada: `v22.x.x` (cualquier 22) y `git version 2.x`. Si `node` no se reconoce o la versión es menor que 22, instalen Node.js 22 LTS desde `nodejs.org`, cierren la terminal y ábrala de nuevo.

3.2 • Instalar las dependencias (solo si no se hizo en la S5)

```
cd ruta/a/laboratorios-evm
npm install
```

Cómo sé que funcionó: termina con una línea del tipo `added NNN packages` y aparece la carpeta `node_modules`. Puede tardar varios minutos en la sala. Las advertencias `npm WARN` no son errores.

3.3 • Comprobar la versión de Hardhat

```
npx hardhat --version
```

Salida esperada: `3.16.0`. Si sale otra, alguien actualizó el `package.json`: no sigan, avisen al docente. Las versiones están congeladas en `versiones.md` y no se cambian a mitad de semestre.

3.4 • Comprobar que el entorno funciona

```
npx hardhat test test/s07/Subasta.test.js
```

Salida esperada, antes de tocar nada: `23 passing`. En el repositorio viene la solución de referencia; así se confirma que el entorno está sano **antes** de empezar a romperlo. La primera vez compila unos 25 contratos: es normal que tarde. Una advertencia sobre `Operaciones.sol` («Function state mutability can be restricted to view») es intencional (Sesión 5) y no es un error.

REGLAS DE SEGURIDAD QUE NO CAMBIAN

Solo red local y Sepolia; nunca dinero real. La frase semilla no se comparte con nadie, tampoco con el docente. No firmen en la billetera nada cuyo contenido no entiendan: antes de confirmar, lean la red, la función y el valor. Estos contratos son material de clase **sin auditar**.

04 El mapa del laboratorio

ARCHIVO	QUÉ ES	PATRÓN QUE ILUSTRA
<code>contracts/s07/Subasta.sol</code>	La subasta completa (solución de referencia).	Ownable, Pausable, retiro, CEI, enum, immutable
<code>andamiaje/s07/Subasta.sol</code>	La misma subasta con 6 TODO. Aquí trabajan.	—
<code>contracts/s07/ISubasta.sol</code>	La interfaz común: estados, eventos y funciones.	interface
<code>contracts/s07/Porcentajes.sol</code>	Porcentajes en puntos básicos.	library, using for
<code>contracts/s07/SubastaPush.sol</code>	Versión ingenua que devuelve al instante. No se modifica.	antipatrón push
<code>contracts/s07/SubastaPullMinima.sol</code>	Idéntica a la push salvo el patrón.	comparación justa de gas
<code>contracts/s07/PostorHostil.sol</code>	El atacante: su <code>receive()</code> revierte.	receive, denegación de servicio
<code>contracts/s07/Empaquetado.sol</code>	Dos structs con los mismos datos en distinto orden.	empaquetado de storage
<code>test/s07/Subasta.test.js</code>	23 pruebas: la especificación. No se modifica.	—
<code>test/s07/GasPushPull.test.js</code>	Escenarios A y B para la tabla de gas.	medir, no suponer

LAS PRUEBAS SON LA ESPECIFICACIÓN

No hay que adivinar qué debe hacer cada función: lo dice la prueba. Cuando una prueba falla, lean su nombre y su código antes de tocar el contrato. El laboratorio está terminado cuando las 23 pasan con **su** versión.

5.1 · Copiar el andamiaje sobre la solución

```
cp andamiaje/s07/Subasta.sol contracts/s07/Subasta.sol      (PowerShell, macOS, Linux)
copy andamiaje\s07\Subasta.sol contracts\s07\Subasta.sol    (cmd de Windows)
npx hardhat test test/s07/Subasta.test.js
```

Salida esperada:

```
7 passing
16 failing
```

Así se empieza, y es lo correcto. Las 7 que ya pasan prueban cosas que no dependen de los TODO: el constructor con duración cero, que solo el dueño pausa (eso lo hace OpenZeppelin), que el contrato rechaza ether directo, la subasta push, la comparación de dos contratos ya hechos y el empaquetado. Si ven **otro** número, o un error de compilación, paren: el andamiaje compila tal cual y se probó que da exactamente 7 y 16. Repitan la copia.

PARA TRABAJAR POR PARTES

El resultado completo abruma. Filtren por el nombre del grupo con `--grep`:

```
npx hardhat test test/s07/Subasta.test.js --grep "pujar"
npx hardhat test test/s07/Subasta.test.js --grep "retirar"
npx hardhat test test/s07/Subasta.test.js --grep "finalizar y estado"
```

Si en algún momento se pierden por completo, recuperen la solución con `git checkout contracts/s07/Subasta.sol` y vuelvan a copiar el andamiaje. Mírenla solo como último recurso: el aprendizaje está en escribirla.

1

TODO 1 · estado()

Calcular en qué estado está la subasta

Qué hace: devuelve `Estado.Finalizada` si ya se llamó `finalizar()`; si no, `Estado.PorFinalizar` si el tiempo llegó a `fin`; si no, `Estado.Abierta`.

Qué escribir: tres líneas con `if` y `return`, en ese orden. El orden importa: una subasta finalizada también tiene el tiempo vencido.

Pista: el plazo se compara con `block.timestamp >= fin` (mayor o igual: en el segundo exacto de `fin` ya no está abierta, igual que en `pujar`).

Cómo sé que funcionó: pasa «recorre los tres estados». Las pruebas comparan con `0n`, `1n` y `2n` porque un `enum` es un entero pequeño (lámina A.4).

2

TODO 2 · minimoSiguiente()

Lo mínimo que debe ofrecer la próxima puja

Qué hace: si nadie ha pujado (`mejorPostor == address(0)`), devuelve `pujaMinima`. Si ya hay postor, devuelve la mejor puja más el 1 %.

Qué escribir: `return mejorPuja + mejorPuja.aplicarBps(INCREMENTO_MINIMO_BPS);` — gracias a `using Porcentajes for uint256`, `aplicarBps` se llama como si fuera un método del número.

Ejemplo trabajado: con `mejorPuja = 2 ETH = 2 · 1018 wei`: $2 \cdot 10^{18} \times 100 / 10\,000 = 2 \cdot 10^{16}$ wei = 0,02 ETH. El mínimo siguiente es 2,02 ETH.

Cómo sé que funcionó: pasan «exige superar la puja anterior al menos en 1 %» y «se usa desde la subasta: 1 % de 2 ETH son 0,02 ETH».

3

TODO 3 · pujar() · CHECKS

Validar antes de tocar nada

Qué hace: revierte con `SubastaCerrada()` si `block.timestamp >= fin` o si finalizada. Después calcula `uint256 minimo = minimoSiguiente();` y revierte con `PujaInsuficiente(minimo, msg.value)` si `msg.value < minimo`.

Detalle que evalúan las pruebas: el error lleva dos argumentos, en ese orden. La prueba lo comprueba con `.withArgs(MINIMA, eth("0.5"))`.

Cómo sé que funcionó: pasan «rechaza una primera puja por debajo del mínimo» y «no acepta pujas después del cierre».

4

TODO 4 · pujar() · EFFECTS

Acreditar, nunca enviar

Qué hace, en este orden:

· Si había mejor postor: `pendientes[mejorPostor] += mejorPuja;` — con `+=`, porque el mismo postor puede ser superado varias veces.

· `mejorPostor = msg.sender; mejorPuja = msg.value;`

· Si `!participo[msg.sender]`: marcarlo en el mapping y hacer `_participantes.push(msg.sender)` (lámina A.2).

· `emit NuevaPuja(msg.sender, msg.value);`

Lo que NO deben hacer: enviar el dinero al superado con `call`. Eso es exactamente la subasta push, y la prueba

★ PULL fallará con `EnvioFallido`.

Cómo sé que funcionó: pasan «acepta la primera puja», «al ser superado, el postor NO recibe su dinero», «registra a cada participante una sola vez», «acumula si el mismo postor es superado varias veces» y ★ PULL.

5

TODO 5 · retirar()

Checks → Effects → Interactions, en el orden exacto

Qué escribir, línea por línea:

```

uint256 monto = pendientes[msg.sender];           // leer
if (monto == 0) revert NadaQueRetirar();          // 1 · checks
pendientes[msg.sender] = 0;                        // 2 · effects
(bool ok, ) = payable(msg.sender).call{value: monto}(""); // 3 · interaction
if (!ok) revert EnvioFallido();
emit Retiro(msg.sender, monto);

```

Por qué el orden importa: si el cero se pusiera **después** del `call`, el receptor podría volver a llamar `retirar()` desde su `receive()` antes de que su saldo quede en cero, y cobrar otra vez (reentrada, lámina A.15; se explota en la Sesión 9). Lo explican en la pregunta 6.

No pongan `whenNotPaused` aquí. Retirar lo propio no se pausa nunca (lámina A.16).

Cómo sé que funcionó: pasan todo el grupo «retirar» y «pausada no acepta pujas, pero Sí permite retirar».

6

TODO 6 · finalizar()

Cerrar sin depender del dueño

Qué hace: revierte con `SubastaAbierta()` si `block.timestamp < fin`; con `YaFinalizada()` si ya se finalizó. Marca `finalizada = true`. Si hubo postor, `pendientes[owner()] += mejorPuja`; — también el beneficiario retira por el patrón pull. Emite `SubastaFinalizada(mejorPostor, mejorPuja)`.

Decisión de diseño: `finalizar()` **no** es `onlyOwner`. Si el dueño pierde su clave, los perdedores igual pueden cerrar y retirar (lámina A.12).

Cómo sé que funcionó: pasa todo el grupo «finalizar y estado», incluida «al final el contrato queda en cero si todos retiran».

06 Punto de control · las 23 en verde

```
npx hardhat test test/s07/Subasta.test.js
```

Salida esperada (se muestra solo el final):

```
S07 · push frente a pull · el postor hostil
✓ ★ contra la subasta PUSH, un postor hostil congela todas las pujas siguientes
✓ ★ contra la subasta PULL, el mismo ataque no bloquea a nadie
  segunda puja · push: 42280 · pull: 55350 · retiro posterior: 29977
✓ gas, comparación justa: pull cuesta MÁS por puja y además exige un retiro aparte

S07 · empaquetado de storage
  desordenado: 88341 gas · ordenado: 66279 gas · ahorro: 22062
✓ ordenar las variables ahorra una ranura: una escritura nueva menos

S07 · biblioteca Porcentajes
✓ se usa desde la subasta: 1 % de 2 ETH son 0,02 ETH

23 passing
```

Cómo sé que terminé: `23 passing` y ningún `failing`. Hagan la captura de pantalla ahora: es el primer ítem de la evidencia. Las cifras impresas deben coincidir exactamente con estas; la red local es determinista.

SI LES DA 22

Casi siempre es la de pausa (pusieron `whenNotPaused` en `retirar`) o la de «acumula» (usaron `=` en lugar de `+=`). Vayan a la sección 13.

07 Parte 2 · El ataque del postor hostil

10 MINUTOS

```
npx hardhat test test/s07/Subasta.test.js --grep "postor hostil"
```

1. **Lean la prueba ★ PUSH** en `test/s07/Subasta.test.js`. El contrato hostil puja 1 ETH en `SubastaPush`. Beto intenta pujar 5 y luego 50 ETH: las dos reversion con `EnvioFallido`, y `mejorPostor` sigue siendo el hostil. Siguen la cadena: `pujar()` de Beto → `call` de devolución al hostil → `receive()` del hostil reierte → `ok == false` → la subasta reierte → la puja de Beto no existe.
2. **Lean la prueba ★ PULL**. Mismo ataque contra su `Subasta`. La puja de Beto entra, y el ETH del hostil queda en `pendientes`: el hostil solo se perjudica a sí mismo (su contrato no tiene cómo llamar `retirar()`).
3. **Rómpalo a propósito**. En su `Subasta.sol`, reemplacen temporalmente la línea `pendientes[mejorPostor] += mejorPuja;` por un envío:

```
(bool ok, ) = payable(mejorPostor).call{value: mejorPuja}("");
if (!ok) revert EnvioFallido();
```

Corran otra vez la suite. **Salida esperada:** falla la ★ PULL con `EnvioFallido` (y también las que dependen de `pendientes`). Acaban de convertir su subasta en la push. **Deshagan el cambio** y confirmen que vuelven las 23.

Anoten la cadena de llamadas: es la pregunta 1 de la reflexión.

08 Parte 3 · Desplegar en Sepolia

10 MINUTOS · PRIMERO ESTO

Se despliega **antes** de la tabla de gas a propósito: el plazo de la subasta (10 minutos) corre mientras trabajan la Parte 4.

1. Abran `remix.ethereum.org`. En el explorador de archivos, creen una carpeta `s07` y dentro **tres** archivos: `ISubasta.sol`, `Porcentajes.sol` y `Subasta.sol`. Peguen en cada uno el contenido de **su** versión. Los tres en la **misma** carpeta: los `import "../ISubasta.sol"` son relativos.
2. Pestaña *Solidity compiler*: versión `0.8.28`, *Advanced configurations* → *Enable optimization* con `200`. Compilen `Subasta.sol`. Los `import` de `@openzeppelin` se descargan solos. **Cómo sé que funcionó:** marca verde en el ícono del compilador.
3. Pestaña *Deploy & run: Environment = Injected Provider – MetaMask*. La billetera debe estar en **Sepolia** y con la cuenta del curso. Contrato: `Subasta`.
4. Parámetros del constructor: `duracionSegundos = 600`, `pujaMinima_ = 10000000000000000` (eso es 0,001 ETH en wei). Pulsen *transact*, lean la ventana de la billetera (red, contrato nuevo, sin valor) y confirmen. **Anoten la dirección del contrato.**
5. **Primera puja** desde la cuenta 1: en el campo *Value* escriban `1` y elijan la unidad `milliether` (o `10000000000000000` en `wei`), luego el botón `pujar`. **Segunda puja** desde la cuenta 2 con `2` milliether. Guarden los dos hashes de transacción.
6. **Cómo sé que funcionó:** `mejorPostor` muestra la cuenta 2, `mejorPuja` muestra `20000000000000000`, `pendientes(cuenta 1)` muestra `10000000000000000`, y `estado` devuelve `0`.

SIN FONDOS DE PRUEBA

El docente reparte ETH de Sepolia desde la billetera institucional. Si la red está caída, despliegan en la *Remix VM* para no perder la práctica y completan la evidencia de Sepolia en casa antes de la entrega.

09 Parte 4 · Ejercicio comparativo de gas: push frente a pull

15 MINUTOS

La prueba `test/s07/GasPushPull.test.js` juega la **misma** subasta contra tres contratos, en dos escenarios:

- **Escenario A:** cuatro postores distintos pujan 1, 2, 3 y 4 ETH. Vence, se finaliza y todos los que tienen saldo pendiente retiran (también el beneficiario).
- **Escenario B:** Ana y Beto se alternan seis pujas (1, 2, 3, 4, 5, 6 ETH). Mismo cierre.

Corran **un escenario a la vez** (`--gas-stats` suma todas las pruebas que corren juntas):

```
npx hardhat test test/s07/GasPushPull.test.js --grep "escenario A" --gas-stats
npx hardhat test test/s07/GasPushPull.test.js --grep "escenario B" --gas-stats
```

Cómo leer la salida. Hay un cuadro por contrato. Cada fila es una función; las columnas `Min`, `Average`, `Median`, `Max` son el gas de sus llamadas y `#calls` cuántas hubo. Ejemplo real del cuadro de `Empaquetado` (de la suite principal):

```
|| contracts/s07/Empaquetado.sol:Empaquetado || | | | | |
|| Function name | Min | Average | Median | Max | #calls ||
|| guardarDesordenado | 88341 | 88341 | 88341 | 88341 | 1 ||
|| guardarOrdenado | 66279 | 66279 | 66279 | 66279 | 1 ||
```

En la subasta, la **primera** puja es la más cara (estrena `mejorPostor` y `mejorPuja`): es el `Max`. La suma de todas las pujas es `Average × #calls`; puede diferir en 1 o 2 unidades de gas del valor exacto porque el promedio se redondea. Además, cada prueba imprime el `TOTAL` exacto del escenario sumando los recibos: úsenlo para **comprobar** su suma, no para copiarla.

LLENAR CON SUS CIFRAS	PUSH · A	PULL MÍNIMA · A	PUSH · B	PULL MÍNIMA · B
pujar · Max (la primera)				
pujar · Min				
pujar · Average × #calls				
finalizar				
retirar · Average × #calls	—		—	
TOTAL del escenario				
pull ÷ push				

Anoten también la fila `pujar` del cuadro de `Subasta.sol` (la completa) en el escenario A.

TRES PREGUNTAS SOBRE LA TABLA (VAN EN EL INFORME)

T1. En el escenario A, ¿cuánto más cuesta pull que push en total, en gas y en porcentaje? ¿Quién paga esa diferencia: el ganador, los perdedores o el beneficiario?

T2. En el escenario B, el `Min` de `pujar` en pull es **menor** que en push. Expliquen por qué con los costos de escritura de storage: escribir en una ranura que estaba en cero cuesta 20 000 de gas; modificar una ranura que ya tenía valor cuesta 2 900 (más 2 100 si es el primer acceso en la transacción, EIP-2929). ¿Qué ranura estrena cada puja pull del escenario A que, en las últimas pujas del B, ya tenía valor?

T3. En el escenario A, las pujas de `Subasta.sol` completa cuestan casi el doble que las de `SubastaPullMinima`. ¿Qué hace de más? ¿Vale la pena? (Pista: lámina A.2.)

10 Parte 5 · Cerrar la subasta en Sepolia

5 MINUTOS

1. Cuando hayan pasado 10 minutos desde el despliegue, lean `estado`: debe devolver `1` (`PorFinalizar`). Si devuelve `0`, esperen: el reloj es el de los bloques de Sepolia, no el de su computador.
2. Llamen `finalizar` desde **cualquier** cuenta. **Cómo sé que funcionó:** `estado` devuelve `2` y en el explorador aparece el evento `SubastaFinalizada`. Guarden el hash.
3. Desde la cuenta 1 (la superada): `retirar`. Desde la cuenta que desplegó (la dueña): `retirar`. Guarden al menos un hash. **Cómo sé que funcionó:** `pendientes` de las dos cuentas vuelve a `0` y el saldo del contrato en el explorador es 0 ETH.
4. **Opcional, recomendado:** verificar el código en `sepolia.etherscan.io` → *Contract* → *Verify and Publish*. Como el contrato importa otros archivos, el formulario de un solo archivo suele fallar: usen la opción *Standard-Json-Input* del explorador o el complemento de verificación de Remix. Compilador `0.8.28` y optimización `200`, exactamente los mismos del despliegue. En la Sesión 8 esto se automatiza con un solo comando.

11 El informe de decisiones de diseño

LA EVIDENCIA QUE PIDE EL PLAN

No es un resumen de lo que hicieron: es la **defensa** de por qué el contrato es como es. Un ingeniero que hereda su código debería poder leerlo y entender qué se decidió, qué se descartó y qué cuesta. Mínimo **cinco decisiones**, en esta tabla:

DECISIÓN	ALTERNATIVA DESCARTADA	POR QUÉ	EVIDENCIA	COSTO
<code>Retiro (pull)</code>	Devolver al superado en la misma puja	Un receptor que revierte congela la subasta push	Pruebas ★ PUSH y ★ PULL	Su cifra de T1
<code>Cero antes del call</code>	Poner en cero después	...	...	...
<code>retirar sin pausa</code>	Pausar todas las funciones	...	...	...
<code>(su decisión 4)</code>				
<code>(su decisión 5)</code>				

Candidatas para las decisiones 4 y 5: `fin` y `pujaMinima` como `immutable`; `finalizar()` sin `onlyOwner`; no declarar `receive()` ni `fallback()`; registrar participantes en un arreglo (¿para qué sirve y qué riesgo tiene?); incremento mínimo del 1 % en puntos básicos; usar `call` en lugar de `transfer`.

LA COLUMNA QUE MÁS PESA

Evidencia. Una decisión sin una prueba que la defienda o una cifra medida que la cuantifique es una opinión. Si una decisión no tiene prueba en la suite, díganlo y describan la prueba que haría falta.

12 Preguntas de reflexión

SE RESPONDEN MIRANDO EL CÓDIGO Y LA SALIDA

Estas preguntas no se pueden responder bien sin haber corrido el laboratorio: piden datos de **su** terminal, de **su** contrato y de **su** despliegue.

1. Escriban la cadena exacta de llamadas por la que la puja de 50 ETH de Beto revierte en la subasta push: qué función llama a cuál, dónde se produce el `revert` y con qué error llega a Beto.
2. ¿Por qué el mismo contrato hostil no bloquea la subasta pull? ¿Quién pierde en ese caso, cuánto, y por qué el contrato hostil no puede recuperarlo?
3. Cuando rompieron su contrato a propósito (sección 7, paso 3), ¿cuántas pruebas fallaron y cuáles? ¿Alguna prueba que esperaban que fallara siguió en verde?
4. El struct ordenado ahorra 22 062 de gas. Muestren con los costos de la sección 9 (T2) que esa cifra corresponde a una ranura nueva menos. ¿Qué otra variable del laboratorio ganaría con un mejor orden, si alguna?
5. Si alguien pone `whenNotPaused` en `retirar()`, ¿qué prueba falla, con qué error, y qué principio de diseño defiende esa prueba?
6. En `retirar()`, ¿qué pasaría si el saldo se pusiera en cero **después** del `call`? ¿Alguna de las 23 pruebas lo detectaría? Describan la prueba que haría falta (qué contrato auxiliar necesitaría y qué comprobaría).

13 Errores frecuentes

CASI TODO SE RESUELVE AQUÍ

SÍNTOMA	CAUSA Y SOLUCIÓN
«npx no se reconoce como comando»	Node.js no está instalado o la terminal se abrió antes de instalarlo. Instalen Node 22 LTS y abran una terminal nueva.
No encuentra <code>@openzeppelin/...</code> o «Cannot find module» al compilar	Falta <code>npm install</code> , o la terminal no está en la carpeta <code>laboratorios-evm</code> .
Al empezar no ven 7 / 16	El andamiaje no se copió sobre <code>contracts/s07/Subasta.sol</code> . Repitan la copia; revisen la ruta.
Falla la de retiro: «changed by 0 wei»	Falta el <code>call</code> : el pendiente se pone en cero pero nunca se envía.
El retiro revierte con <code>NadaQueRetirar</code> la primera vez	Pusieron el pendiente en cero antes de leer el monto: el monto leído es 0. Primero se lee, después se pone en cero.
Falla «acumula si es superado varias veces»	<code>=</code> en lugar de <code>+=</code> al acreditar al superado: se pierde lo acreditado antes.
<code>EnforcedPause</code> al retirar	<code>whenNotPaused</code> en <code>retirar()</code> . Quítenlo: retirar lo propio no se pausa.
El dueño no puede retirar tras <code>finalizar</code>	<code>finalizar()</code> no acredita <code>mejorPuja</code> al <code>owner()</code> .
No acepta la primera puja de 1 ETH	<code>minimoSiguiente()</code> aplica el 1 % aunque no haya postor. Sin postor, devuelve <code>pujaMinima</code> tal cual.
La ★ PULL falla con <code>EnvioFallido</code>	Devolvieron el dinero con <code>call</code> dentro de <code>pujar()</code> : esa es la push.
<code>PujaInsuficiente</code> pasa pero «withArgs» falla	Los argumentos del error van en orden <code>(minimo, msg.value)</code> .
La tabla de <code>--gas-stats</code> trae cifras mezcladas	Corrieron los dos escenarios juntos. Usen <code>--grep "escenario A"</code> y luego <code>--grep "escenario B"</code> .
Remix: «not found ISubasta.sol»	Los tres archivos deben estar en la misma carpeta de Remix.
Remix: «Gas estimation failed» al pujar	El <i>Value</i> está en cero o por debajo del mínimo, o la subasta ya venció. Revisen la unidad (wei, gwei, milliether).
<code>finalizar</code> revierte en Sepolia	Aún no pasan los 600 s según el reloj de los bloques. Esperen un minuto y lean <code>estado</code> .

14 Lista de chequeo antes de entregar

- ☐ Captura con `23 passing` usando su versión de `Subasta.sol`.
- ☐ Probaron romper su contrato (sección 7) y lo dejaron como estaba.
- ☐ Tabla de gas completa, con los dos escenarios, y respuestas T1, T2 y T3.
- ☐ Dirección del contrato en Sepolia.
- ☐ Hashes de: dos pujas, la finalización y al menos un retiro.
- ☐ Informe de decisiones de diseño con al menos cinco decisiones, cada una con evidencia.
- ☐ Las seis preguntas de reflexión respondidas con datos propios.
- ☐ Ninguna frase semilla, clave privada ni captura que la muestre en el documento.
- ☐ Nombres de los dos integrantes en la primera página.

15 Rúbrica

LABORATORIO 07 · SEGUNDO CORTE

CRITERIO	PESO	EXCELENTE (5,0)	ACEPTABLE (3,5)	INSUFICIENTE (<3,0)
Subasta completa	20 %	23 en verde; código limpio y comentado; CEI exacto.	23 en verde con código confuso o redundante.	Menos de 23 en verde.
Despliegue en Sepolia	20 %	Dirección y los cuatro hashes; subasta finalizada y retirada; código verificado.	Dirección y hashes incompletos.	Sin evidencia en cadena: no se califica.
Tabla de gas	20 %	Completa, sumas correctas, T1–T3 explicadas con costos de storage.	Completa con explicaciones superficiales.	Incompleta o cifras inventadas.
Informe de decisiones	25 %	Cinco o más decisiones, cada una con alternativa, razón, evidencia y costo.	Cinco decisiones, alguna sin evidencia.	Menos de cinco, o sin alternativas.
Reflexión	15 %	Las seis, con datos de su propia salida.	Las seis, genéricas.	Faltan respuestas.

16 Trabajo autónomo · refactorizar los certificados de la S6

PARA LA SESIÓN
8

El plan pide refactorizar el contrato de certificados de la Sesión 6 (`contracts/s06/CertificadosUSB.sol`) aplicando **dos patrones** de hoy. No hay una sola respuesta correcta: se califica que cada patrón resuelva un riesgo real del contrato y que lo justifiquen.

PATRÓN CANDIDATO	QUÉ RIESGO DE <code>CertificadosUSB</code> ATACA	LA TRAMPA A EVITAR
Ownable2Step	<code>cambiarEmisor</code> traspasa el control en un paso: un error de tipeo pierde el contrato para siempre.	Reemplazar <code>soloEmisor</code> por <code>onlyOwner</code> cambia el error (<code>NoEsEmisor</code> → <code>OwnableUnauthorizedAccount</code>): las pruebas de la S6 que lo esperan fallarán. Explíquenlo.
Pausable	Si roban la clave del emisor, el ladrón puede emitir diplomas falsos.	¿Se pausa <code>revocar</code> ? Pausar la revocación impediría anular justo los diplomas falsos. <code>verificar</code> nunca se pausa.
Empaquetado	El struct <code>Certificado</code> tiene <code>string</code> , <code>uint64</code> y dos <code>bool</code> .	Midan antes y después con <code>--gas-stats</code> sobre <code>test/s06</code> : si no cambia, díganlo; no inventen el ahorro.

Entrega (en la Sesión 8): el contrato refactorizado, una tabla con los dos patrones (qué cambió, qué riesgo mitiga, qué cuesta en gas, medido) y la salida de `npx hardhat test test/s06/CertificadosUSB.test.js` contra su versión, explicando cada prueba que falle y por qué. Trabajen en una copia (`contracts/s06/CertificadosUSBv2.sol`) y no modifiquen el original.

LA PREGUNTA DE LA SEMANA

El postor hostil no robó nada y aun así ganó. ¿Dónde más —en el curso o en el mundo— alguien bloquea a todos solo con negarse a cooperar? Tráiganla pensada: abre la Sesión 8.

17 Referencias

- Documentación de Solidity 0.8.28 · *Layout of State Variables in Storage*.
https://docs.soliditylang.org/en/v0.8.28/internals/layout_in_storage.html
- OpenZeppelin Contracts 5.x · *Access Control* (Ownable, Ownable2Step) y *Utils* (Pausable).
<https://docs.openzeppelin.com/contracts/5.x/api/access>
- EIP-1884 · *Repricing for trie-size-dependent opcodes* (SLOAD de 200 a 800).
<https://eips.ethereum.org/EIPS/eip-1884>
- EIP-2929 · *Gas cost increases for state access opcodes* (2 100 en frío, 2 900 para modificar).
<https://eips.ethereum.org/EIPS/eip-2929>
- ConsenSys Diligence (2019). *Stop Using Solidity's transfer() Now*.

- King of the Ether Throne · *Post-Mortem Investigation* (febrero de 2016).

<https://www.kingoftheether.com/postmortem.html>